DESIGN PACKAGE / 01

From field report to coordinated response.

System architecture, database schema visualization, and web / mobile frontend prototypes for the existing UP Manila project.

SCRIBER

Disaster Casualty Management System

01 / System architecture

Vector diagram, client and API boundaries, authentication, data services, offline queue and realtime paths.

02 / Database schema

Core ERD, extension map, full searchable table catalog with source DDL, and read-only inspection queries.

03 / UI / UX prototype

Clickable web and mobile flows, low-fidelity wireframe mode, offline queue simulation, and review decisions.

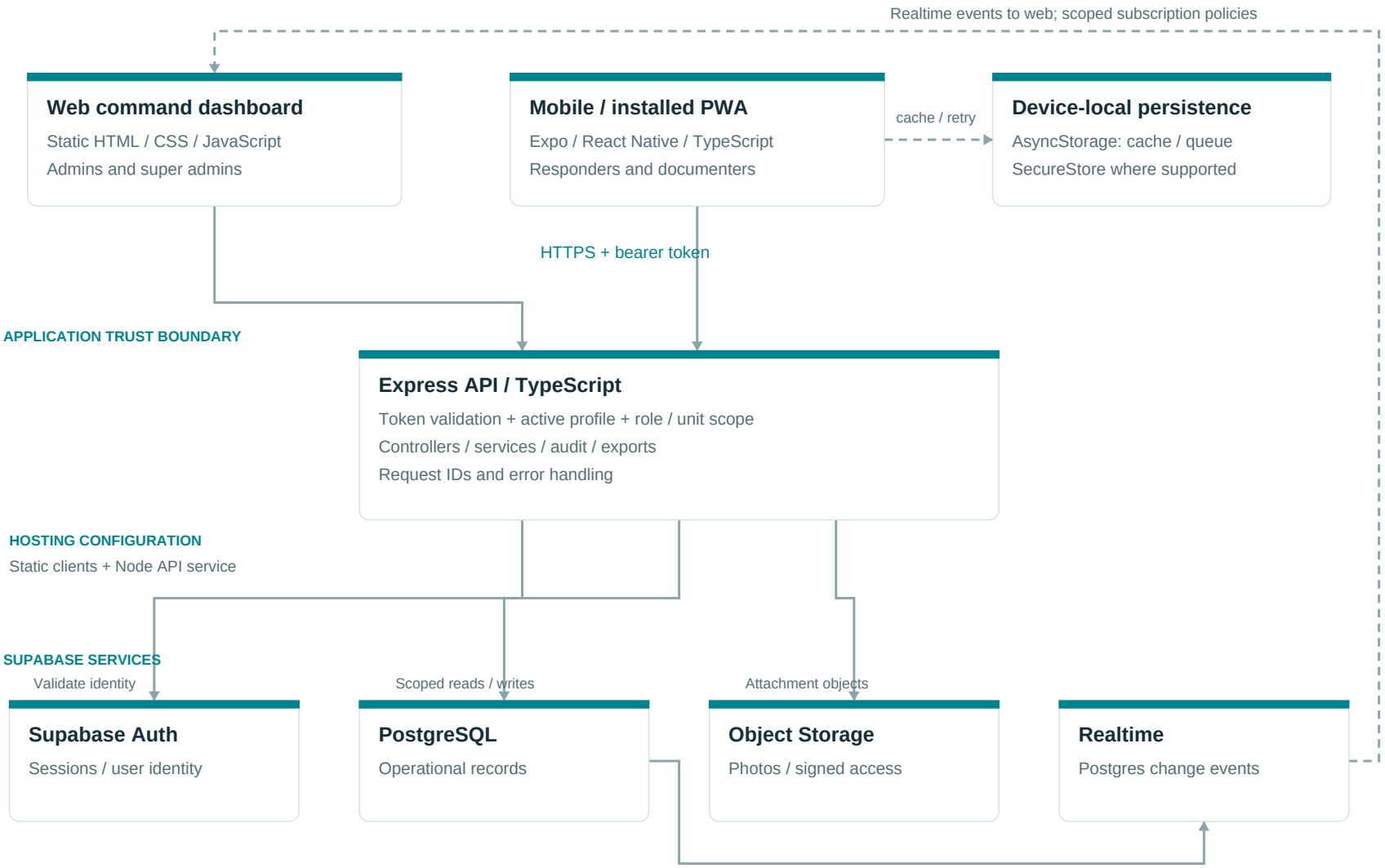
REVIEW STATUS

Prepared for stakeholder review. No approval is implied. Reviewer and decision fields are provided on the final page.

Basis: repository source only. No production data, live database inspection, deployment verification, or clinical protocol validation was performed.

Two clients, one operational backend.

Solid lines show request / data paths. Dashed lines show local persistence or client event subscriptions.



Solid: request / data path. Dashed: client-side cache or event channel. No direct client database CRUD is shown.

Source: api/src/app.ts; api/src/middleware/auth.ts; website/app.js; mobile/src/offline/casualtyQueue.ts.

The incident record connects the care journey.

Selected key fields and links. Relationship optionality follows the reviewed declarations; confirm against the deployed catalog.

PK = primary key; FK = foreign key; UQ = unique. Every report belongs to one incident, one casualty, and one encoder.

Relationship view is selective. Full field declarations and all source tables are in the searchable schema catalog.

Source: UP-Manila-DB.sql and api/sql/*.sql. The linked HTML catalog preserves exact source statements and filenames.

Care history and incident operations.

Optional extensions are grouped by their declared parent. Grouping does not assert that every table is deployed or actively used.

casualty_incidents / report extensions

FK casualty_incident_id -> casualty_incidents.id

casualty_status_history
0..N status transitions

casualty_treatments
0..N treatment entries

clinical_procedures
0..N procedures; optional facility_id

icu_encounters
0..N ICU episodes; optional facility_id

casualty_outcomes
0..1 final outcome; UNIQUE report FK

casualty_case_links
0..1 link per report; case_id is a UUID group

incidents / operational extensions

FK incident_id -> incidents.id

incident_response_timelines
0..1 timeline; UNIQUE incident FK

dmmp_staff_call_downs
0..N call-down / staffing entries

medical_coordination_assessments
0..1 evaluation; UNIQUE incident FK

continuity_of_care_assessments
0..1 assessment; UNIQUE incident FK

facility_resource_snapshots
0..N snapshots; optional facility_id

sitreps / ems_vehicle_arrivals
0..N situation reports / vehicle arrivals

User-linked support tables

form_drafts, notifications, audit_logs, call_down_staff

Additional incident / user links

responder_safety_reports, responder_safety_responses
incident_ic_backups, incident_ic_delegations

Reference groups summarize declared FK links; they do not imply that every optional extension is deployed or used by every workflow.

Data ownership and integrity.

Design interpretation of the source model; the full declared field list and source DDL are included in schema-catalog.json and the HTML explorer.

users	Application profile; id links to auth.users. Active status, role and reporting context influence API access.
incidents	Operational scope; created_by identifies the permanent owner. Acting IC delegation uses separate tables.
casualties	Identity attributes. Keep the person distinct from the incident report; victim code semantics require explicit review.
casualty_incidents	Report hub linking casualty, incident, encoder and optional destinations. Root SQL declares client_record_id UNIQUE NOT NULL.
casualty_case_links	Groups related reporting-role records by case_id. UNIQUE casualty_incident_id; UNIQUE (case_id, role). No separate cases table is declared here.
triage / transport / care	Multiple assessments, movements and encounters preserve the care timeline. Optional facility links differ across tables.
history / attachments	Verification and status histories provide traceability. Attachments link report metadata to storage objects.

Important distinction: the source field union is a discovery aid, not an executable baseline or a live database export.

32 tables found in the reviewed SQL.

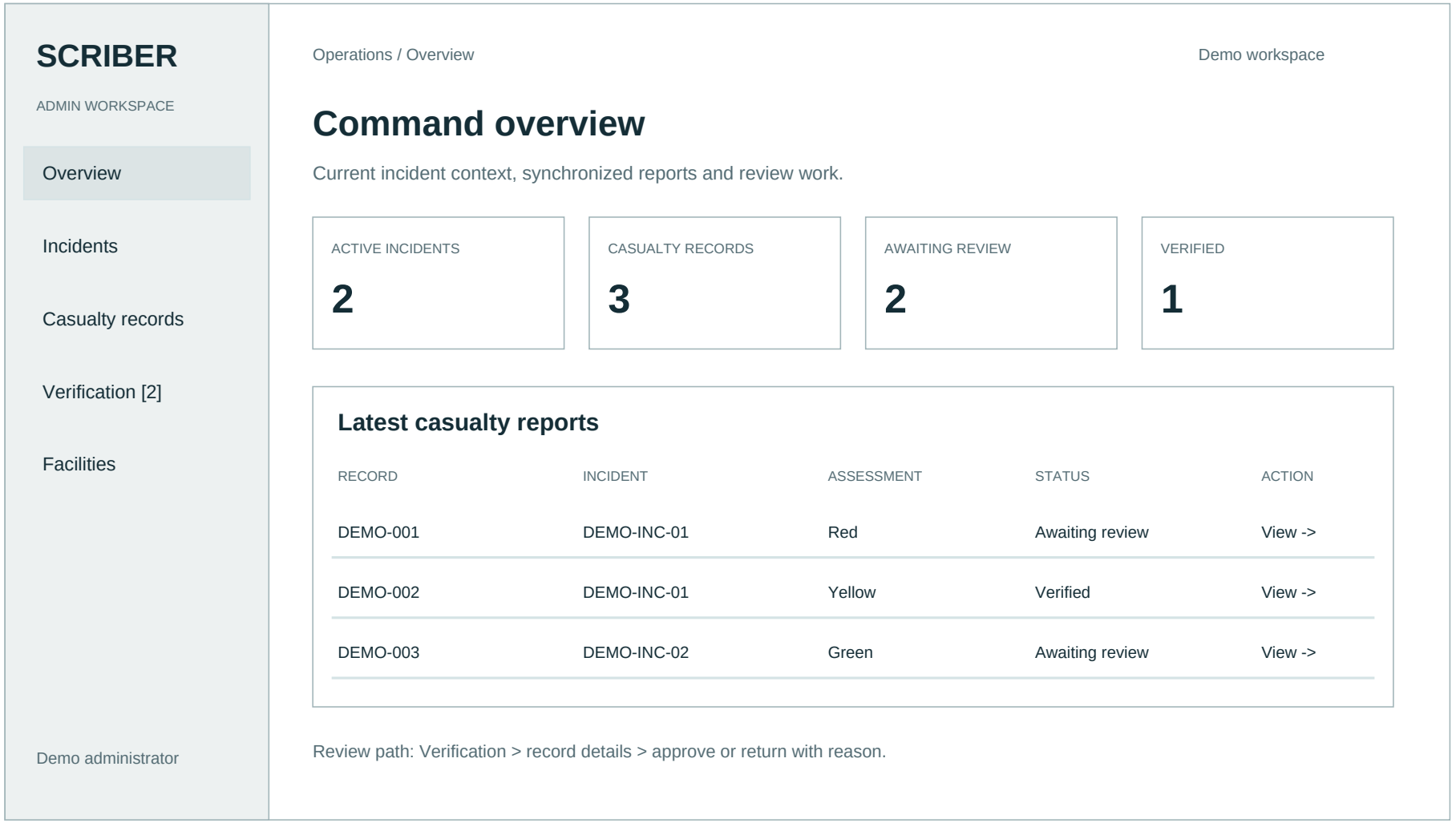
Every table below is linked to exact CREATE TABLE / ADD COLUMN source declarations in the companion explorer. Counts are source field unions.

attachments	9 fields
audit_logs	14 fields
call_down_staff	11 fields
casualties	22 fields
casualty_case_links	9 fields
casualty_incidents	25 fields
casualty_outcomes	12 fields
casualty_status_history	14 fields
casualty_transport_records	16 fields
casualty_treatments	10 fields
casualty_triage_assessments	16 fields
casualty_verification_history	7 fields
clinical_procedures	8 fields
continuity_of_care_assessments	8 fields
dmmp_staff_call_downs	15 fields
ems_vehicle_arrivals	7 fields
evacuation_centers	15 fields
facility_encounters	32 fields
facility_resource_snapshots	13 fields
form_drafts	8 fields
healthcare_facilities	15 fields
icu_encounters	9 fields
incident_ic_backups	7 fields
incident_ic_delegations	11 fields
incident_response_timelines	22 fields
incidents	21 fields
medical_coordination_assessments	10 fields
notifications	10 fields
responder_safety_reports	12 fields
responder_safety_responses	10 fields
sitreps	14 fields
users	14 fields

Scope: root UP-Manila-DB.sql plus all api/sql/*.sql files. Migration ordering, drops, renames and type changes are not reconstructed.

A command workspace for review and coordination.

Desktop information hierarchy: incident scope, operational totals, latest reports, and a visible verification queue.

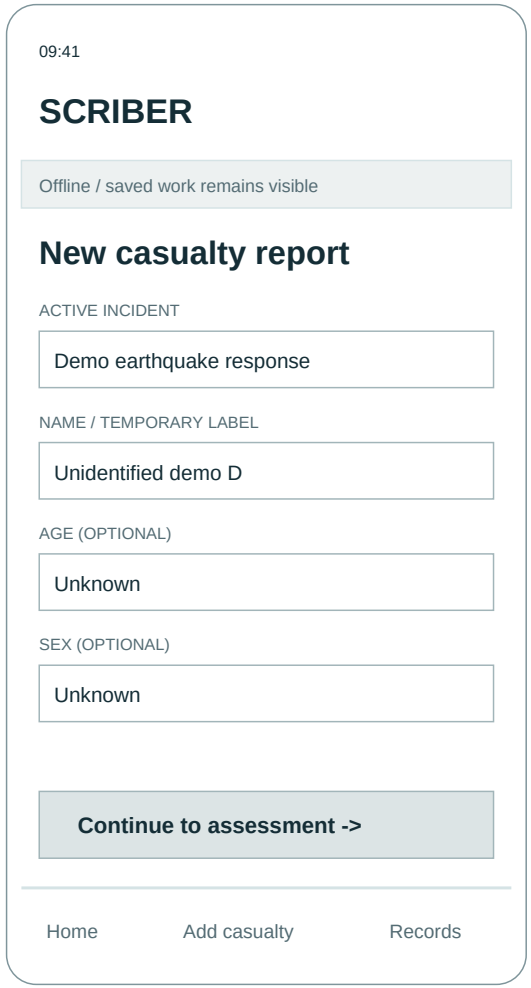


Desktop layout wireframe. The HTML prototype includes filters, incident reports, record detail, verification and the mobile handoff.

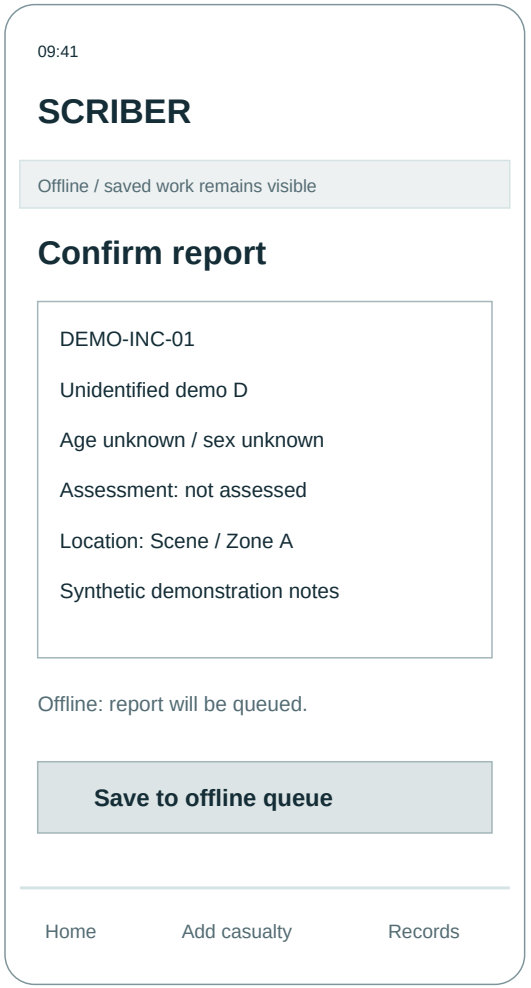
Short steps. Explicit submission state.

Identity -> assessment -> review -> submission or offline queue. The interactive prototype includes the assessment step and both connection states.

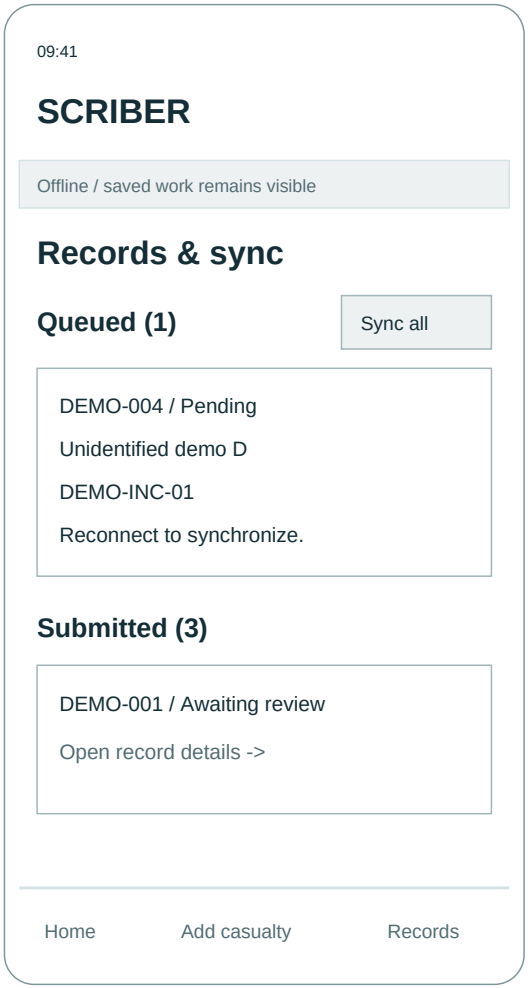
01 / IDENTITY



02 / REVIEW



03 / QUEUE



Touch targets: at least 44 px in the HTML design. Queue, connection state and review feedback use text as well as color.

Walk through the complete reporting loop.

The prototype uses synthetic records in memory. It makes no backend requests and does not persist records after reloading the page.

1. Capture	Mobile > Add casualty. Select an active incident, enter a name or unidentified label, and complete the assessment step.
2. Review	Check the report before submitting. Back navigation retains draft fields. Age is optional; entered values must be 0 to 130.
3. Queue	Enable Offline before submission. The report appears as Pending in Records and remains absent from the web register.
4. Synchronize	Reconnect and select Sync all. Queued reports become submitted and appear in the web verification list.
5. Verify	Web > Verification > Review. Approve the record or return it with a required reason. Status and history update in both views.
6. Inspect / reset	Search and filter the register, inspect record history, toggle Wireframe view, or Reset demo to restore the starting data.

UI design: cyan / neutral palette aligned to the existing project; textual state labels; visible focus outlines; responsive layouts; native form controls.

Simulated: authentication, sync transport and offline storage. Photo capture and full role-specific clinical forms remain in the existing application.

Resolve source drift before implementation sign-off.

These are review findings, not changes applied to your database or application.

Roles / baseline	The root user_role enum lists the original roles, while API middleware supports additional roles such as field_responder, sa_responder and documenter. Compare the live role column / enum with add-separated-responder-roles.sql and the application guards.
Migration provenance	Multiple files declare overlapping tables and constraints. The source union cannot establish migration order or which definitions are deployed. Export the live catalog with schema-review.sql.
Identifiers / retries	Root SQL declares a unique client_record_id. The API checks it before submission and calls a transaction function. Confirm the deployed uniqueness guarantee and concurrent retry behavior.
Victim codes	replace-global-casualty-id-number-uniqueness.sql drops global identity-code uniqueness. API checks codes within an incident; a database-level incident/code guarantee remains a review decision.
Authorization / realtime	Review API unit / role scope and Realtime policies together. Supabase service-role calls need explicit API authorization. Client filtering is insufficient for sensitive record visibility.
Offline recovery	Confirm shared-device queue isolation, attachment size limits, partial-upload retry, and expired-session behavior on actual devices and weak connections.

Reviewable now. Approval is a recorded decision.

Complete the architecture, database and UI walkthrough with the appropriate project reviewers.

[ ] Architecture: client responsibilities, API boundary, hosting ownership and data paths accepted.

[ ] Database: live schema compared with source; enum, role and migration differences reconciled.

[ ] Identity: victim-code and retry uniqueness guarantees agreed and verified.

[ ] UI / UX: desktop, mobile, empty states, queue workflow and return-for-correction feedback reviewed.

[ ] Operational roles: field, stabilization, facility and command terminology confirmed.

[ ] Validation: API permissions, Realtime visibility and real-device offline recovery assigned for testing.

Reviewer: _____ Role / organization: _____

Decision: Approve / Approve with changes / Revise Date: _____

Conditions / comments: _____

_____

Companion files: index.html, editable SVG / Mermaid diagrams, schema-catalog.json, schema-review.sql, and README.md.